Chapter 11: PRIORITY_QUEUE (Heap)

This is one of the most important STL containers for interviews.

Used in:

- Kth Largest Element
- Kth Smallest Element
- Top K Frequent Elements
- Dijkstra Algorithm
- Merge K Sorted Lists
- Scheduling Problems
- Heap Problems

What is Priority Queue?

Normal Queue:

```
10 20 30 40

Pop -> 10
```

FIFO.

Priority Queue:

```
10 20 30 40

Pop -> 40
```

Highest priority comes first.

By default:

```
priority_queue<int> pq;
```

is a:

```
MAX HEAP
```

Header

```
#include <queue>
```

Declaration

Max Heap

```
priority_queue<int> pq;
```

Insert

push()

```
pq.push(10);
pq.push(30);
pq.push(20);
```

Heap:

```
30
20
10
```

Access Largest Element

top()

```
cout << pq.top();
```

Output:

```
30
```

Remove Largest

pop()

```
pq.pop();
```

Now:

```
20
10
```

Size

```
pq.size();
```

Empty

```
pq.empty();
```

Complete Example

```
priority_queue<int> pq;

pq.push(10);
pq.push(30);
pq.push(20);

cout << pq.top() << endl;
```

```
pq.pop();

cout << pq.top();
```

Output:

```
30
20
```

Complexity

Operation	Complexity
push	O(log n)
pop	O(log n)
top	O(1)
size	O(1)
empty	O(1)

Max Heap Example

Input:

```
10 5 20 15
```

Insertion:

```
20
15
10
5
```

Pop sequence:

```
20
15
```

```
10
5
```

Largest first.

Min Heap

Very important.

Declaration:

```
priority_queue<
    int,
    vector<int>,
    greater<int>
> pq;
```

Insert:

```
pq.push(10);
pq.push(30);
pq.push(20);
```

Top:

```
cout << pq.top();
```

Output:

```
10
```

Smallest element.

Max Heap vs Min Heap

Max Heap

```
priority_queue<int> pq;
```

Top:

```
Largest
```

Min Heap

```
priority_queue<
int,
vector<int>,
greater<int>
> pq;
```

Top:

```
Smallest
```

Priority Queue of Pair

Extremely important.

Declaration

```
priority_queue<pair<int,int>> pq;
```

Insert

```
pq.push({10,1});
pq.push({30,2});
pq.push({20,3});
```

Top:

```
cout<<pq.top().first;
```

Output:

```
30
```

Comparison Rule

Pairs compare:

```
first
then second
```

Same as sorting pairs.

Min Heap of Pair

```
priority_queue<
pair<int,int>,
vector<pair<int,int>>,
greater<pair<int,int>>
> pq;
```

Interview favorite.

Used in:

```
Dijkstra
Shortest Path
Graphs
```

Custom Comparator

Advanced.

Example:

Sort by second value.

```
class Compare
{
public:
    bool operator()(
        pair<int,int> a,
        pair<int,int> b
    )
    {
        return a.second < b.second;
    }
};
```

Use:

```
priority_queue<
pair<int,int>,
vector<pair<int,int>>,
Compare
> pq;
```

Important for interviews.

Common Interview Problem 1

Kth Largest Element

Input:

```
3 2 1 5 6 4

k=2
```

Output:

```
5
```

Solution:

Min Heap of size k.

Complexity:

```
O(n log k)
```

LeetCode #215

Common Interview Problem 2

Kth Smallest Element

Use:

```
max heap
```

of size k.

Common Interview Problem 3

Top K Frequent Elements

Input:

```
1 1 1 2 2 3

k=2
```

Output:

```
1 2
```

Uses:

```
unordered_map
+
```

```
priority_queue
```

LeetCode #347

Common Interview Problem 4

Merge K Sorted Arrays

Uses:

```
min heap
```

Complexity:

```
O(n log k)
```

Common Interview Problem 5

Dijkstra Algorithm

Uses:

```
priority_queue<
pair<int,int>,
vector<pair<int,int>>,
greater<pair<int,int>>
>
```

One of the most important graph algorithms.

Printing Priority Queue

No iterators.

Use copy.

```
auto temp = pq;

while(!temp.empty())
{
    cout<<temp.top()<<" ";

    temp.pop();
}
```

Swap

```
pq1.swap(pq2);
```

Emplace

```
pq.emplace(10);
```

Better than:

```
pq.push(10);
```

for objects.

Practice Questions

Easy

Q1

Insert:

```
10 20 5 30
```

Print elements in priority order.

Q2

Find largest element.

Q3

Find smallest element using min heap.

Q4

Print heap size.

Q5

Delete largest element.

Medium

Q6

Find kth largest element.

Q7

Find kth smallest element.

Q8

Sort array using priority queue.

(Hint: Heap Sort idea)

Q9

Find top 3 largest numbers.

Q10

Merge two sorted arrays using heap.

Interview Level

Q11

Kth Largest Element

LeetCode #215

Q12

Top K Frequent Elements

LeetCode #347

Q13

Merge K Sorted Lists

LeetCode #23

Q14

Find Median from Data Stream

LeetCode #295

Q15

Dijkstra Algorithm

Common Mistakes

Mistake 1

Thinking priority_queue is min heap.

Default is:

```
MAX HEAP
```

Mistake 2

Trying:

```
pq[0]
```

❌ Not allowed.

Mistake 3

Trying iterators.

```
pq.begin()
```

❌ Not available.

Mistake 4

Calling:

```
pq.top()
```

on empty heap.

❌ Runtime Error.

Revision Sheet

Header

```
#include <queue>
```

Max Heap

```
priority_queue<int> pq;
```

Min Heap

```
priority_queue<
int,
vector<int>,
greater<int>
> pq;
```

Insert

```
pq.push(x);
pq.emplace(x);
```

Remove

```
pq.pop();
```

Access

```
pq.top();
```

Utilities

```
pq.size();
pq.empty();
pq.swap();
```

Complexity

```
push O(log n)
pop  O(log n)
top  O(1)
```

Mental Trigger

When you see:

- Kth Largest
- Kth Smallest
- Top K
- Highest Priority
- Scheduling
- Dijkstra
- Median Stream

Think:

```
PRIORITY_QUEUE
```
